

CodeColosseum

Software Requirements Specification

March 28, 2026

Kerem Adalı 150123055

Buğra Kaya 150123045

Ali Talip Keleş 150122029

Prepared for

CSE3044 Software Engineering Term Project

Marmara University, Spring 2025–2026

Contents

1	Introduction	3
1.1	Purpose.....	3
1.2	Scope.....	3
1.3	Definitions, Acronyms, and Abbreviations.....	4
1.4	References	5
1.5	Overview.....	5
2	General Description	5
2.1	Product Perspective	5
2.2	Product Functions.....	6
2.3	Stakeholders and User Characteristics	7
2.3.1	Stakeholders	7
2.3.2	User Roles	7
2.4	General Constraints.....	7
2.5	Assumptions and Dependencies.....	8
3	Specific Requirements	8
3.1	External Interface Requirements	9
3.1.1	User Interfaces	9
3.1.2	Hardware Interfaces.....	9
3.1.3	Software Interfaces	9
3.1.4	Communications Interfaces.....	10
3.2	Functional Requirements	10
3.2.1	FR-1: User Registration and Authentication	10
3.2.2	FR-2: Matchmaking	11
3.2.3	FR-3: Match Session	12
3.2.4	FR-4: Automated Code Evaluation	13
3.2.5	FR-5: Rating System	15
3.2.6	FR-6: Problem Proposal and Approval Workflow.....	15
3.2.7	FR-7: Leaderboard and History.....	17
3.3	Non-Functional Requirements	18
3.3.1	Performance	18
3.3.2	Reliability.....	18
3.3.3	Availability.....	18
3.3.4	Security	18
3.3.5	Maintainability.....	19
3.3.6	Portability	19
3.4	Inverse Requirements	19
3.5	Design Constraints	19
3.6	Logical Database Requirements	20
3.7	Other Requirements	20
4	Use Cases	21
4.1	System Overview	21
4.2	User Registration and Authentication (FR-1)	23
4.2.1	Diagram	23
4.2.2	Written Use Cases	23

4.3	Matchmaking (FR-2)	25
4.3.1	Diagram	25
4.3.2	Written Use Case.....	25
4.4	Match Session and Code Evaluation (FR-3, FR-4)	26
4.4.1	Diagram	26
4.4.2	Written Use Case.....	27
4.5	Rating System (FR-5)	28
4.5.1	Diagram	28
4.5.2	Written Use Case.....	29
4.6	Problem Proposal and Approval Workflow (FR-6).....	30
4.6.1	Diagram	30
4.6.2	Written Use Case.....	31
4.7	Leaderboard and Match History (FR-7).....	32
4.7.1	Diagram	32
4.7.2	Written Use Cases	33
5	System Sequence Diagram	35
A	Appendices	35
A.1	Decision Log: Code Execution Strategy	36
A.2	Decision Log: Rating Algorithm	37
A.3	Decision Log: Match Winner Determination.....	38

1 Introduction

The following subsections provide an overview of the entire Software Requirements Specification (SRS) for CodeColosseum.

1.1 Purpose

This SRS defines the complete software requirements for CodeColosseum, a real-time 1v1 competitive programming platform intended to serve the competitive programming community as a commercial web product.

Competitive programming platforms such as Codeforces and LeetCode are well-established, but they are oriented toward asynchronous, solo problem solving: a user picks a problem, submits a solution at their own pace, and compares their result against a static leaderboard after the fact. There is no mechanism for two users to compete directly against each other on the same problem at the same moment. CodeColosseum addresses this gap by providing a dedicated synchronous 1v1 format in which two contestants are matched by skill level, given the same problem simultaneously, and compete to produce the first accepted solution within a fixed time window. This format introduces an element of real-time pressure and direct rivalry that is absent from existing platforms and is well suited to the short, high-intensity sessions that university students and developers prefer for competitive practice.

The document specifies the system's intended behavior in sufficient detail to support all phases of development, from architectural design through implementation and testing, and to serve as the contractual baseline against which the delivered system will be evaluated.

The intended audience for this document is as follows. The development team uses this document as the authoritative reference for all design and implementation decisions; any feature or behavior not specified here is considered out of scope. The system administrator relies on this document to understand platform management responsibilities, including the problem approval workflow and user oversight. Contestants, the primary end users of the platform, are represented throughout this document as the principal stakeholders whose experience and competitive fairness the system is designed to serve.

1.2 Scope

The software product described in this document is named **CodeColosseum**.

CodeColosseum is a web-based competitive programming platform in which two users compete head-to-head in real time by solving the same algorithmic problem simultaneously. The system will:

- Enable registered users to enter a matchmaking queue and be automatically paired against an opponent of similar skill level based on an Elo rating.
- Deliver the same programming problem to both matched users at the same instant and start a shared countdown timer.
- Accept code submissions from users, evaluate them automatically against predefined test cases within an isolated execution environment, and return a verdict.

- Determine a match winner based on submission correctness and speed, and update both users' ratings accordingly.
- Provide a community-driven problem contribution workflow in which any registered user may propose a problem for administrator review and approval.
- Maintain a global leaderboard and per-user match history.

The primary goal of CodeColosseum is to provide a fair, engaging, and educationally stimulating environment for competitive algorithmic problem solving. The system is intentionally scoped for a one-semester academic project and will not include advanced features such as team-based contests, real-time in-match chat, plagiarism detection, or spectator modes.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer; an architectural style for web APIs
JWT	JSON Web Token; a compact, self-contained token used for authentication
Elo	A method for calculating the relative skill levels of players, originally designed for chess and adapted here for rating contestants
K-factor	A constant used in the Elo formula that controls how much a single match result affects a player's rating
Judge	The subsystem responsible for compiling and executing submitted code against test cases and returning a verdict
Sandbox	An isolated execution environment that prevents submitted code from accessing or affecting the host system or other processes
AC	Accepted — a submission that passes all test cases
WA	Wrong Answer — a submission that fails at least one test case
TLE	Time Limit Exceeded — a submission whose execution time exceeds the problem's defined limit
MLE	Memory Limit Exceeded — a submission whose memory usage exceeds the problem's defined limit
CE	Compilation Error — a submission that fails to compile
RE	Runtime Error — a submission that crashes during execution
PENDING	The status assigned to a problem proposal awaiting administrator review
APPROVED	The status assigned to a problem proposal that has been accepted and published to the problem pool
REJECTED	The status assigned to a problem proposal that has been declined by an administrator

1.4 References

1. IEEE Std 830-1984, *IEEE Guide to Software Requirements Specifications*, Institute of Electrical and Electronics Engineers, 1984.
2. Judgeo API Documentation. Available at: <https://judgeo.com>
3. Spring Boot Reference Documentation, Pivotal Software. Available at: <https://spring.io/projects/spring-boot>
4. React Documentation. Available at: <https://react.dev>
5. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs/>
6. E. W. Elo, *The Rating of Chessplayers, Past and Present*, Arco Publishing, New York, 1978. (Background reference for the Elo algorithm.)

1.5 Overview

The remainder of this SRS is organized as follows.

Section 2 (General Description) provides background context for the product: its perspective relative to other systems, a high-level summary of its functions, characteristics of its users, general constraints on the development, and assumptions and dependencies that affect the requirements.

Section 3 (Specific Requirements) is the primary technical section. It specifies all external interface requirements, functional requirements (organized by feature), non-functional requirements, inverse requirements, design constraints, logical database requirements, and other requirements.

Section 4 (UML Diagrams) presents use case diagrams for each functional area alongside written use case descriptions that detail actor interactions, preconditions, and exception flows.

The Appendix contains decision log entries documenting key engineering decisions made during requirements analysis.

2 General Description

This section describes the general factors that affect CodeColosseum and its requirements. It does not state specific requirements; rather, it provides the context needed to understand them.

2.1 Product Perspective

CodeColosseum is an independent, self-contained web application. It is not a component of a larger organizational system and has no dependencies on any institutional infrastructure.

The product is inspired by existing competitive programming platforms such as Codeforces (<https://codeforces.com>) and LeetCode (<https://leetcode.com>), but is differentiated by its exclusive focus on synchronous 1v1 real-time matchmaking rather than

asynchronous solo problem solving. Unlike those platforms, CodeColosseum is not intended for large-scale public deployment; it is scoped for a contained user base in an academic context.

The principal external interfaces of CodeColosseum are as follows:

- **Spring Boot Backend:** The backend is a Spring Boot application running on Java. It serves as the application server, hosting all business logic, REST API endpoints, WebSocket handlers, matchmaking coordination, and rating calculations. It acts as the bridge between the React frontend, the PostgreSQL database, and the Judgeo API, and no component communicates with another without passing through the backend.
- **Judgeo API:** An external code execution service. CodeColosseum delegates all compilation and sandboxed code execution to this service, interacting with it exclusively through its documented REST API over HTTPS. The Judgeo service is treated as a black box; its internal implementation is outside the boundary of this product.
- **PostgreSQL Database Server:** The persistent data store for all application data. It is a separate process deployed as part of the CodeColosseum deployment environment and is not shared with any other application.
- **React Frontend:** A client-side single-page application served by the Spring Boot backend. It communicates with the backend exclusively through a defined REST API (over HTTP/HTTPS) and a WebSocket interface for real-time match state updates.

No other external system depends on CodeColosseum. The system has no direct hardware interface requirements beyond the standard network-connected devices used by its users.

2.2 Product Functions

At a high level, CodeColosseum performs the following functions:

- **User account management:** Registration, login, profile display, and role-based access control.
- **Matchmaking:** Placing users in a queue and pairing them with opponents of similar Elo rating.
- **Match session management:** Delivering a shared problem to both users simultaneously, managing a countdown timer, and accepting submissions during the match window.
- **Automated code evaluation:** Forwarding submissions to the Judgeo API, collecting verdicts, and presenting results to users.
- **Match resolution and rating update:** Determining the match winner based on defined rules and recalculating both players' Elo ratings using the standard Elo formula.

- **Problem management:** Allowing contestants to propose problems, and allowing administrators to review, edit, approve, or reject proposals.
- **Leaderboard and history:** Displaying a global ranking of users by Elo rating and providing each user with access to their personal match and submission history.

2.3 Stakeholders and User Characteristics

2.3.1 Stakeholders

The following parties have an interest in the development and operation of CodeColosseum.

Stakeholder	Interest
Contestants	The primary end users. Interested in fair matchmaking, accurate rating calculation, a responsive code editor, and timely verdict feedback. Their experience and competitive fairness are the principal design drivers.
Administrators	Responsible for maintaining problem quality. Interested in an efficient review workflow, the ability to edit proposals before approval, and auditability of all decisions.
Development Team	Responsible for designing, implementing, and testing the system within one academic semester. Interested in clear, unambiguous requirements and a well-scoped feature set.
Course Evaluators	Assess the project against the CSE3044 term project rubric. Interested in correct application of software engineering practices, completeness of documentation, and a working demonstration.

2.3.2 User Roles

CodeColosseum has two distinct user roles.

Contestant. A registered user who participates in matches, submits solutions, proposes problems, and tracks their Elo rating over time. The typical contestant is a university student or software developer with working knowledge of at least one of the supported programming languages (Java, Python, or C++) and basic familiarity with algorithmic problem solving. Contestants are assumed to be comfortable using a browser-based code editor and interpreting compiler or judge feedback. No special administrative knowledge is required for this role.

Administrator. A privileged user responsible for maintaining the platform. The administrator reviews community-submitted problem proposals, verifies their correctness, edits them as necessary, and approves or rejects them for publication. Administrators are expected to have sufficient technical expertise to evaluate the quality, solvability, and correctness of algorithmic problems, including the ability to assess whether test cases are adequate.

2.4 General Constraints

The following items constrain the developer's options for designing the system:

- **External service dependency:** The system depends on the continuous availability of the JudgeO API for code execution. Downtime or rate-limiting of this service will degrade match functionality. This dependency is accepted as a deliberate design trade-off (see Appendix A.1).
- **Project timeline:** The system must be fully designed, implemented, and tested within one academic semester by a team of three developers. This constraint directly limits the scope of features included.
- **Technology selection:** The backend must be implemented in Java using the Spring Boot framework, and the frontend must be implemented using React. PostgreSQL must be used as the relational database. These decisions are fixed and are not open to revision.
- **Supported languages:** The set of programming languages available to contestants is constrained by the language support offered by the JudgeO API. At minimum, Java, Python, and C++ must be supported.
- **Security requirement:** No user-submitted code may be executed on the application server. All code execution must occur within the JudgeO sandbox.
- **Browser compatibility:** The system must function correctly on current versions of Chrome, Firefox, and Edge without browser-specific customization.

2.5 Assumptions and Dependencies

The requirements stated in this SRS are based on the following assumptions. If any of these assumptions prove incorrect, the requirements may need to be revised.

- It is assumed that the JudgeO API will remain publicly accessible and free to use within the submission volume required by this project throughout the development and evaluation period.
- It is assumed that all users access the platform via a modern web browser with JavaScript enabled. The system is not required to support non-JavaScript environments or legacy browsers.
- It is assumed that a PostgreSQL server (version 14 or later) will be available and accessible from the application server throughout development and evaluation.
- It is assumed that the Elo rating system is an acceptable proxy for player skill in this context, and that a K-factor of 32 is appropriate given the expected volume of matches per user.

3 Specific Requirements

This section specifies all requirements in sufficient detail for a software engineer to design and implement CodeColosseum. Each requirement is stated in measurable, verifiable, and unambiguous terms in accordance with IEEE Std 830-1984.

3.1 External Interface Requirements

3.1.1 User Interfaces

The system shall provide a browser-based graphical user interface. The following screens shall be included:

- **Registration screen:** Fields for username, email address, and password, with inline validation feedback.
- **Login screen:** Fields for email and password; a generic error message on failure (without disclosing whether the email exists).
- **Dashboard:** Displays the user's current Elo rating, a summary of recent matches, a button to enter the matchmaking queue, and a link to the leaderboard.
- **Match screen:** A split-panel layout with the problem statement (including constraints and sample input/output) on the left, and a code editor with language selection and a submit button on the right. A countdown timer is displayed prominently. The verdict of the most recent submission is shown in real time.
- **Post-match screen:** Displays the match outcome (win/loss/ draw), both contestants' Elo changes, and a summary of submissions made during the match.
- **Leaderboard screen:** A paginated table of all users ranked by Elo rating in descending order, showing username, rating, and number of matches played.
- **Problem proposal form:** Fields for problem title, statement, input/output format, constraints, time limit, memory limit, and test case pairs.
- **Administrator panel:** A list of pending problem proposals with options to view, edit, approve, or reject each one.

The interface shall be responsive and usable on standard desktop and laptop screen sizes (minimum 1024px width). Mobile support is not required.

3.1.2 Hardware Interfaces

CodeColosseum has no direct hardware interface requirements. It is a server-side web application accessed through standard network-connected computing devices. No special hardware is required beyond what is standard for web application hosting.

3.1.3 Software Interfaces

- **Judgeo API**
 - *Purpose:* Compilation and sandboxed execution of contestant code submissions.
 - *Interface:* REST API over HTTPS. Requests are submitted as JSON payloads containing the source code, language identifier, standard input, and resource limits. Responses contain the execution verdict, standard output, standard error, and resource usage statistics.

- *Version*: The system shall use the Judgeo v2 API.

- **PostgreSQL**

- *Purpose*: Persistent storage for all application data.
- *Interface*: Database connection managed through JDBC. The application communicates with the database via object-relational mapping.
- *Version*: PostgreSQL 14 or later.

- **React Frontend**

- *Purpose*: Browser-side user interface.
- *Interface*: The frontend communicates with the Spring Boot backend via two interfaces: (1) a RESTful API over HTTP/HTTPS for standard request-response interactions, and (2) a WebSocket connection for real-time match state synchronization (timer, opponent submission status, match outcome).

3.1.4 Communications Interfaces

- All HTTP communication between the frontend and backend shall use HTTPS in production.
- Authentication state shall be conveyed via JWT tokens transmitted in the HTTP Authorization header using the Bearer scheme.
- Real-time match state synchronization (countdown timer, opponent verdict notification, match termination) shall be implemented using the WebSocket protocol (RFC 6455).
- Communication with the Judgeo API shall occur over HTTPS from the backend server only. The frontend shall never communicate directly with Judgeo.

3.2 Functional Requirements

3.2.1 FR-1: User Registration and Authentication

FR-1.1 Introduction. The system shall allow users to create accounts and authenticate themselves. Authentication is the gateway to all other system functionality; unauthenticated users may only view the registration and login screens.

FR-1.2 Inputs.

- Registration: username (string, 3–20 characters), email address (valid RFC 5322 format), password (string, minimum 8 characters).
- Login: email address, password.

FR-1.3 Processing.

- On registration, the system shall validate that the email address is not already associated with an existing account and that the username is not already taken.

If either condition fails, registration shall be rejected with a specific informative message.

- The password shall be hashed using bcrypt with a cost factor of at least 10 before being stored. The plaintext password shall not be persisted anywhere.
- On login, the system shall verify the submitted password against the stored bcrypt hash. On success, a JWT token with a configurable expiry (default: 24 hours) shall be issued and returned to the client.
- A newly registered account shall be assigned the **CONTESTANT** role and a default Elo rating of 1000.

FR-1.4 Outputs.

- Successful registration: a new user record is created; the user is redirected to the login screen with a confirmation message.
- Successful login: a JWT token is returned to the frontend and stored client-side; the user is redirected to the dashboard.

FR-1.5 Constraints.

- Usernames must be unique system-wide and between 3 and 20 characters (alphanumeric and underscores only).
- Passwords must be at least 8 characters.
- Email addresses must be unique system-wide.

FR-1.6 Error Handling.

- Duplicate email or username shall return HTTP 409 with a descriptive error message.
- Invalid login credentials shall return HTTP 401 with a generic message (e.g., “Invalid email or password”) to avoid disclosing whether the email exists in the system.
- Malformed input (e.g., invalid email format) shall be rejected at the frontend before submission and at the backend with HTTP 400.

3.2.2 FR-2: Matchmaking

FR-2.1 Introduction. The system shall provide an automated matchmaking mechanism that pairs logged-in contestants with opponents of comparable skill. Matching is based on Elo rating proximity to ensure competitive fairness.

FR-2.2 Inputs. User action to join the matchmaking queue (a single button click on the dashboard). The user’s current Elo rating is retrieved from the database and used implicitly by the matchmaking algorithm.

FR-2.3 Processing.

- When a user joins the queue, the system shall add them to an in-memory queue with their Elo rating and a timestamp recording when they joined.
- The system shall continuously scan the queue for pairs of users every 1000ms (1 second) whose Elo ratings differ by no more than a configurable threshold (default: 200 points). When such a pair is found, a match session shall be created and both users shall be notified via their WebSocket connections.
- A user already in the queue or in an active match shall not be permitted to enter the queue again.

FR-2.4 Outputs.

- Both matched users receive a WebSocket notification containing the match session identifier and are redirected to the match screen.
- A match record is created in the database with status ACTIVE, recording both participant user IDs, the assigned problem ID, and the match start timestamp.

FR-2.5 Constraints.

- A user may not be in the matchmaking queue and in an active match simultaneously.
- A user may not be matched against themselves.

FR-2.6 Error Handling.

- If one of the two matched users disconnects before the match screen loads (within a 10-second grace period), the match shall be cancelled, the remaining user shall be returned to the queue, and the disconnecting user's session shall be terminated.
- If a user attempts to join the queue while already queued or in a match, the request shall be rejected with HTTP 409.

3.2.3 FR-3: Match Session

FR-3.1 Introduction. A match session is the core interaction unit of CodeColosseum. During a session, two matched contestants receive the same problem and compete to produce the first accepted solution within a fixed time window.

FR-3.2 Inputs.

- Source code entered by the contestant in the browser-based editor.
- Selected programming language (from the supported language list).
- Submission action (button click).

FR-3.3 Processing.

- Upon match creation, the system shall select a problem from the approved problem pool. Problem selection shall exclude problems that either contestant has encountered in a previous match. If there is no such problem, then the matchmaking is cancelled with no elo change for both contestants.

- The same problem shall be delivered to both contestants simultaneously via Web-Socket. The match timer shall begin at the moment of delivery.
- Each submission shall be forwarded to the Code Execution component (FR-4) for evaluation. The result shall be recorded and displayed to the submitting user.
- The first contestant to receive an AC verdict wins the match immediately; the match session is closed.
- If the timer expires without an AC verdict from either contestant, the contestant who passed the greater number of test cases across all their submissions (the submission with the greatest number of passing cases is taken into count) wins. If both passed the same number, the match is recorded as a draw.
- After the match concludes, the rating update (FR-5) shall be triggered automatically.

FR-3.4 Outputs.

- Each submission verdict is displayed to the submitting user in real time (within 10 seconds per NFR-1).
- Upon match conclusion, both users receive a WebSocket notification with the match outcome and are redirected to the post-match screen.
- The match record in the database is updated to COMPLETED with the winner, loser (or draw), and end timestamp.

FR-3.5 Constraints.

- Match duration is fixed at 30 minutes.
- A contestant may submit multiple times during a match; there is no submission count limit.
- A contestant shall not be able to view their opponent's submitted code or verdict during the match.

FR-3.6 Error Handling.

- If the Judgeo API is unavailable when a submission is received, the submission shall fail with a user-visible error message ("Execution service temporarily unavailable"). The match shall not be voided; the user may resubmit when the service recovers.
- If a contestant disconnects during a match and does not reconnect within 5 minutes, they shall forfeit the match. The remaining contestant is declared the winner.

3.2.4 FR-4: Automated Code Evaluation

FR-4.1 Introduction. The system shall automatically evaluate each code submission by executing it against all test cases defined for the problem. Evaluation is delegated to the Judgeo API and performed in an isolated sandbox environment.

FR-4.2 Inputs.

- Source code (text string).
- Programming language identifier (Judgeo language ID).
- Problem time limit (in seconds) and memory limit (in megabytes).
- Test case set: a list of (standard input, expected standard output) pairs stored in the database for the problem.

FR-4.3 Processing.

- The system shall submit each test case to the Judgeo API as a separate execution request, passing the source code, language identifier, the test case's standard input, and the problem's time and memory limits.
- Each test case is evaluated independently. A test case is considered passed if and only if the program's standard output (trimmed of trailing whitespace) exactly matches the expected output.
- The overall verdict is AC if and only if all test cases are passed. Otherwise, the verdict is the first non-AC result encountered (WA, TLE, MLE, RE, or CE).
- The number of test cases passed is recorded for use in FR-3 tie resolution.

FR-4.4 Outputs.

- A verdict code (AC, WA, TLE, MLE, RE, or CE).
- The number of test cases passed out of the total.
- For CE: the compiler error message, returned to the user.
- For RE: a runtime error indicator (without exposing system internals).
- The submission record is saved to the database with the verdict, timestamp, and match association.

FR-4.5 Constraints.

- Time and memory limits are defined per problem by the approving administrator and are passed unmodified to Judgeo.
- No user-submitted code shall be executed on the application server. All execution occurs within the Judgeo sandbox.

FR-4.6 Error Handling.

- If the Judgeo API returns an unexpected error code or times out, the submission shall be marked as JUDGE_ERROR and the user shall be notified. The submission shall not be counted as a test case pass or as an AC verdict.

3.2.5 FR-5: Rating System

FR-5.1 Introduction. The system shall maintain an Elo rating for each contestant and update both contestants' ratings after every completed match. The rating system provides the foundation for fair matchmaking and the global leaderboard.

FR-5.2 Inputs.

- Match outcome: WIN, LOSS, or DRAW from the perspective of each contestant.
- Current Elo ratings of both contestants, retrieved from the database at the time the match concludes.

FR-5.3 Processing. The system shall apply the standard Elo formula. For contestant A with rating R_A competing against contestant B with rating R_B :

- Expected score for A : $E_A = 1 / (1 + 10^{(R_B - R_A)/400})$
- New rating for A : $R'_A = R_A + K \cdot (S_A - E_A)$

where $K = 32$ (fixed K-factor) and S_A is the actual score (1 for a win, 0.5 for a draw, 0 for a loss). The same formula is applied symmetrically for contestant B .

The rating update shall be applied atomically within a single database transaction covering both contestants.

FR-5.4 Outputs.

- Updated Elo ratings for both contestants, persisted to the database.
- Rating delta (change in points) for each contestant, included in the post-match screen payload.

FR-5.5 Constraints.

- A contestant's Elo rating shall not fall below 0 as a result of any single match.
- Rating updates shall occur only after the match record has been finalized as COMPLETED.

FR-5.6 Error Handling.

- If the database transaction for rating updates fails, it shall be retried up to three times before the error is logged and flagged for manual review. The match record shall remain COMPLETED regardless of the rating update outcome.

3.2.6 FR-6: Problem Proposal and Approval Workflow

FR-6.1 Introduction. The system shall support a community-driven problem contribution model. Any registered contestant may submit a problem proposal. Administrators review proposals and decide whether to approve (publish) or reject them. This workflow ensures problem quality while allowing community participation.

FR-6.2 Inputs.

- Problem title (string, 5–100 characters).
- Problem statement (free text, supporting plain text formatting).
- Input format description and output format description.
- Constraints section (describing valid input ranges).
- Time limit (integer, 1–10 seconds).
- Memory limit (integer, 16–512 megabytes).
- At least one input/output test case pair (the proposer may submit multiple).
- Suggested difficulty tag: Easy, Medium, or Hard (optional; administrator may override).
- Administrator inputs for review: approval/rejection decision, optional edits to any of the above fields, and an optional rejection reason message.

FR-6.3 Processing.

- Upon submission, the proposal is stored in the database with status PENDING and associated with the proposing user's ID.
- Administrators may view a list of all PENDING proposals, sorted by submission date.
- An administrator may edit any field of a PENDING proposal before making a decision.
- If approved, the proposal status is set to APPROVED and the problem becomes immediately available in the active problem pool for use in matchmaking.
- If rejected, the proposal status is set to REJECTED. An optional rejection reason is stored and communicated to the proposer.
- A problem must have at least three test case pairs to be eligible for approval; the system shall enforce this at the approval step.

FR-6.4 Outputs.

- The proposing user receives an in-system notification of the approval or rejection decision (visible on their dashboard).
- Approved problems are added to the problem pool and become eligible for matchmaking problem selection.

FR-6.5 Constraints.

- Only users with the ADMIN role may approve or reject proposals.
- A problem with fewer than three test cases shall not be approvable; the approve action shall be disabled in the administrator panel until at least three test cases are present.

FR-6.6 Error Handling.

- Proposals with missing required fields (title, statement, input format, output format, time limit, memory limit, or at least one test case) shall be rejected at the frontend with inline validation messages before submission.
- Attempts by non-administrator users to access the approval endpoints shall be rejected with HTTP 403.

3.2.7 FR-7: Leaderboard and History

FR-7.1 Introduction. The system shall provide users with visibility into global standings and personal performance history. This supports the competitive motivation of the platform and allows users to track their progress over time.

FR-7.2 Inputs.

- No user input is required to view the leaderboard.
- For match history: the requesting user's identity (from their JWT token). Pagination parameters (page number, page size) are optional query parameters.

FR-7.3 Processing.

- The leaderboard is generated by querying all users ordered by current Elo rating in descending order. The result is paginated (default: 50 users per page).
- Leaderboard query results will be cached for 60 seconds to reduce database read load.
- Match history is retrieved by filtering the match records table by the requesting user's ID. Each record includes the opponent's username, the problem title, the outcome (win/loss/draw), the rating change, and the match date.

FR-7.4 Outputs.

- A paginated leaderboard response containing: rank, username, current Elo rating, and total matches played.
- A paginated match history response containing: opponent username, problem title, outcome, Elo delta, and match timestamp.

FR-7.5 Constraints.

- The leaderboard is publicly accessible (does not require authentication).
- Match history is private; a user may only view their own history.

FR-7.6 Error Handling.

- If data cannot be retrieved due to a database error, the endpoint shall return HTTP 503 and an appropriate error message shall be displayed to the user.
- Requests for another user's match history shall be rejected with HTTP 403.

3.3 Non-Functional Requirements

3.3.1 Performance

- **NFR-1 (Verdict latency):** 95% of submission verdicts shall be returned to the user within 10 seconds of submission when the number of concurrent active matches does not exceed 20 and the Judgeo API response time is within its documented SLA. When the number of concurrent active matches exceeds 20, verdict latency may degrade but the system shall continue to process all submissions and shall not drop or discard any submission request. Latency is measured from the moment the user clicks the submit button to the moment the verdict is displayed on the match screen.
- **NFR-2 (Page load):** The leaderboard and dashboard pages shall load within 2 seconds for up to 100 concurrent users.
- **NFR-3 (Matchmaking latency):** Under normal usage (at least 10 users in the queue), two compatible users shall be matched within 60 seconds in expectation.

3.3.2 Reliability

- **NFR-4 (Rating consistency):** Rating updates shall be applied within a single atomic database transaction. No match result shall be permanently lost or produce an inconsistent rating state due to a partial failure.
- **NFR-5 (Judge fault tolerance):** A Judgeo API timeout or error on a single submission shall not crash or invalidate the match session. The system shall recover gracefully and allow the user to resubmit.

3.3.3 Availability

- **NFR-6:** The system is intended for academic demonstration and evaluation purposes. No formal uptime SLA is defined. However, the system shall remain stable and operational throughout all scheduled project evaluation and presentation periods.

3.3.4 Security

- **NFR-7 (Password storage):** All user passwords shall be hashed using bcrypt with a minimum cost factor of 10. Plaintext passwords shall never be logged, stored, or transmitted.
- **NFR-8 (Authentication):** All API endpoints that expose non-public data or permit state-changing operations shall require a valid JWT token. Requests with missing or invalid tokens shall be rejected with HTTP 401.
- **NFR-9 (Authorization):** Endpoints restricted to administrators shall verify the ADMIN role claim in the JWT and reject requests from users without that role with HTTP 403.
- **NFR-10 (Code execution isolation):** No user-submitted code shall be executed on the application server or in any environment outside of the Judgeo sandbox.

3.3.5 Maintainability

- **NFR-11 (Execution abstraction):** The code execution logic shall be encapsulated behind a JudgeService interface. The Judgeo API shall be one concrete implementation of this interface. Replacing Judgeo with an alternative execution backend shall require changes only to that implementation class, with no changes to any other component.
- **NFR-12 (Layered architecture):** The backend shall follow a three-layer architecture: controller layer (HTTP handling), service layer (business logic), and repository layer (data access). Business logic shall not appear in controller or repository classes.

3.3.6 Portability

- **NFR-13:** The backend application shall run on any environment providing Java 17 or later and network access to a PostgreSQL instance and the Judgeo API endpoint.
- **NFR-14:** The frontend shall function correctly on the current stable release of Chrome, Firefox, and Edge, without any browser-specific CSS or JavaScript customization.

3.4 Inverse Requirements

The following requirements describe what the system shall *not* do.

- The system shall not execute user-submitted code directly on the application server under any circumstances.
- The system shall not expose one contestant's source code, submission history, or verdict to their opponent at any point during an active match.
- The system shall not allow a user to participate in more than one active match simultaneously.
- The system shall not publish a problem to the active problem pool without explicit ADMIN-role approval.
- The system shall not retain plaintext passwords in any storage, log, or transmission channel.

3.5 Design Constraints

- **DC-1 (Backend framework):** The backend shall be implemented in Java 17 using the Spring Boot 3.x framework.
- **DC-2 (Frontend framework):** The frontend shall be implemented using React (version 18 or later).
- **DC-3 (Database):** PostgreSQL (version 14 or later) shall be used as the relational database. No other database engine is permitted as the primary store.

- **DC-4 (Code execution):** In the initial release, all code execution shall be delegated to the Judgeo API. Direct execution of user code on the application server is prohibited.
- **DC-5 (Team and timeline):** The system must be fully implementable and testable by a three-person development team within one academic semester (approximately 14 weeks of active development). Any feature that cannot realistically be completed within this constraint shall be excluded from scope.

3.6 Logical Database Requirements

The system requires a relational database with the following logical structure. Data integrity shall be enforced via foreign key constraints and appropriate uniqueness constraints.

- **User:** Stores username (unique), email address (unique), bcrypt-hashed password, role (CONTESTANT or ADMIN), current Elo rating, account creation timestamp, and total matches played.
- **Problem:** Stores title, problem statement, input format, output format, constraints, time limit (seconds), memory limit (MB), difficulty tag, approval status (PENDING / APPROVED / REJECTED), the ID of the proposing user, the ID of the reviewing administrator (if reviewed), and the review timestamp.
- **TestCase:** Stores a standard input string, an expected output string, and a foreign key reference to its parent Problem. Each TestCase is only accessible server-side and shall never be returned to contestants.
- **Match:** Records the IDs of both contestants (foreign keys to User), the ID of the problem used, the match status (ACTIVE / COMPLETED / CANCELLED), the outcome (WIN_A / WIN_B / DRAW), and the start and end timestamps.
- **Submission:** Records the submitting user ID, match ID, programming language, source code text, verdict, number of test cases passed, and submission timestamp.
- **RatingHistory:** Records each rating change event, including the user ID, the match ID, the rating before the match, the rating after the match, and the timestamp. This table supports the display of rating progression over time.

Match, Submission, and RatingHistory records shall be retained indefinitely as part of each user's permanent history. Problem proposals with REJECTED status shall also be retained for administrative audit purposes.

3.7 Other Requirements

- **OR-1 (Supported languages):** The system shall support at minimum the following programming languages at launch: Java, Python 3, and C++17. Additional languages supported by Judgeo may be enabled by an administrator without code changes.
- **OR-2 (Default rating):** All newly registered users shall be assigned an initial Elo rating of 1000.

- **OR-3 (Difficulty tagging):** Each approved problem shall be tagged with a difficulty level (Easy, Medium, or Hard), assigned or confirmed by the approving administrator.
- **OR-4 (Problem pool size):** The active problem pool shall contain at least 10 approved problems before the matchmaking feature is made available to users.

4 Use Cases

This section presents the use case diagrams and written use case descriptions for Code-Colosseum, organized by functional area. Each diagram corresponds to one or more functional requirements defined in Section 3.2.

Two primary actor types appear across all diagrams. The **Contestant** is any registered user who participates in matches. The **Administrator** is a privileged user responsible for problem management. A **System** actor represents automated behavior triggered internally with no direct user action. A **Guest** actor represents an unauthenticated visitor.

The following UML use case notation conventions are used throughout. An «include» relationship indicates that the base use case unconditionally incorporates the behavior of the included use case as part of its normal flow. An «extend» relationship indicates that the extending use case adds optional or conditional behavior to the base use case at a defined extension point. Actor–use case associations are shown as solid lines.

4.1 System Overview

Figure 1 presents the high-level view of all actors and their primary interactions with the system. It serves as a map to the detailed diagrams and written use cases that follow.

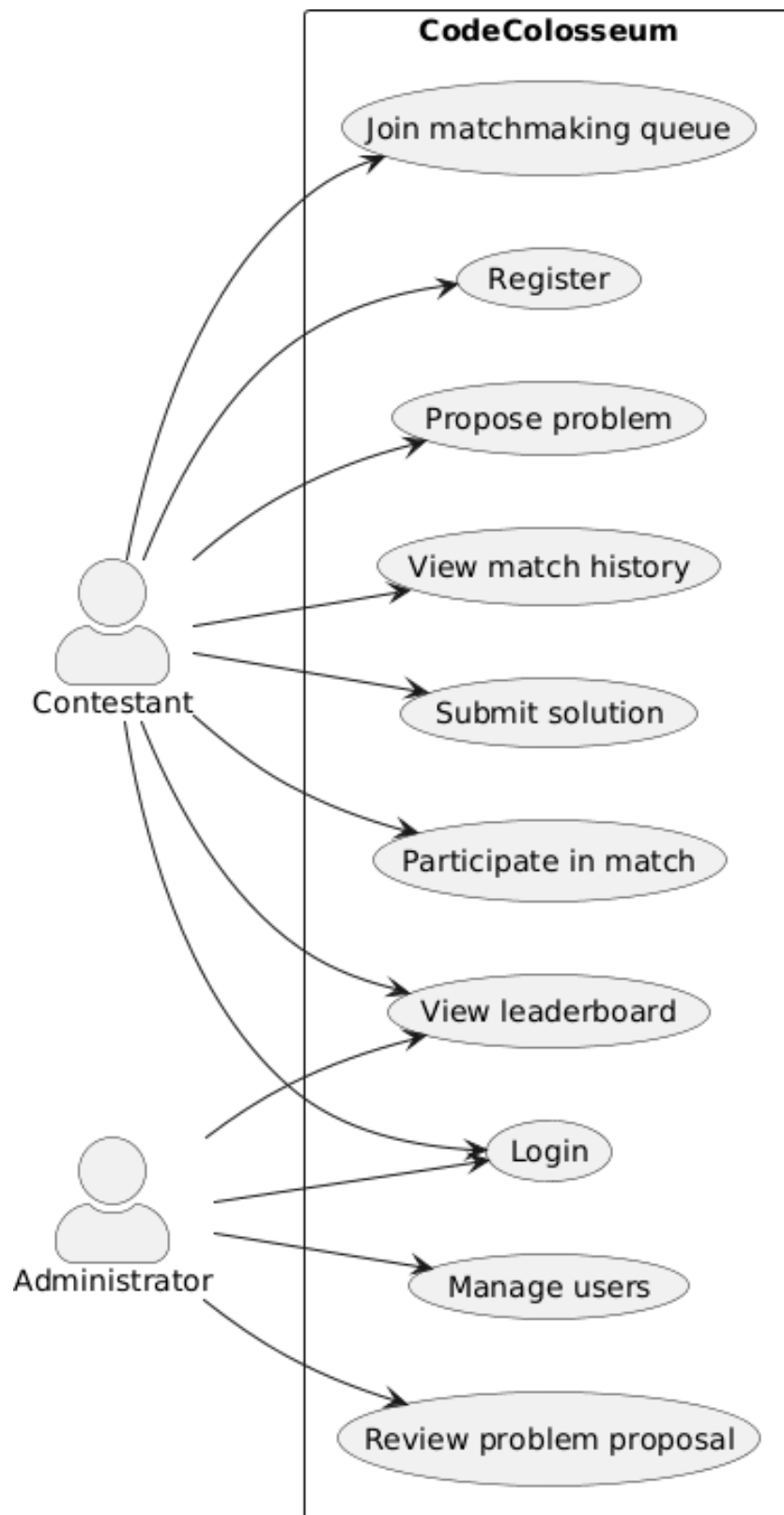


Figure 1: Use case overview — all actors and primary use cases

4.2 User Registration and Authentication (FR-1)

4.2.1 Diagram

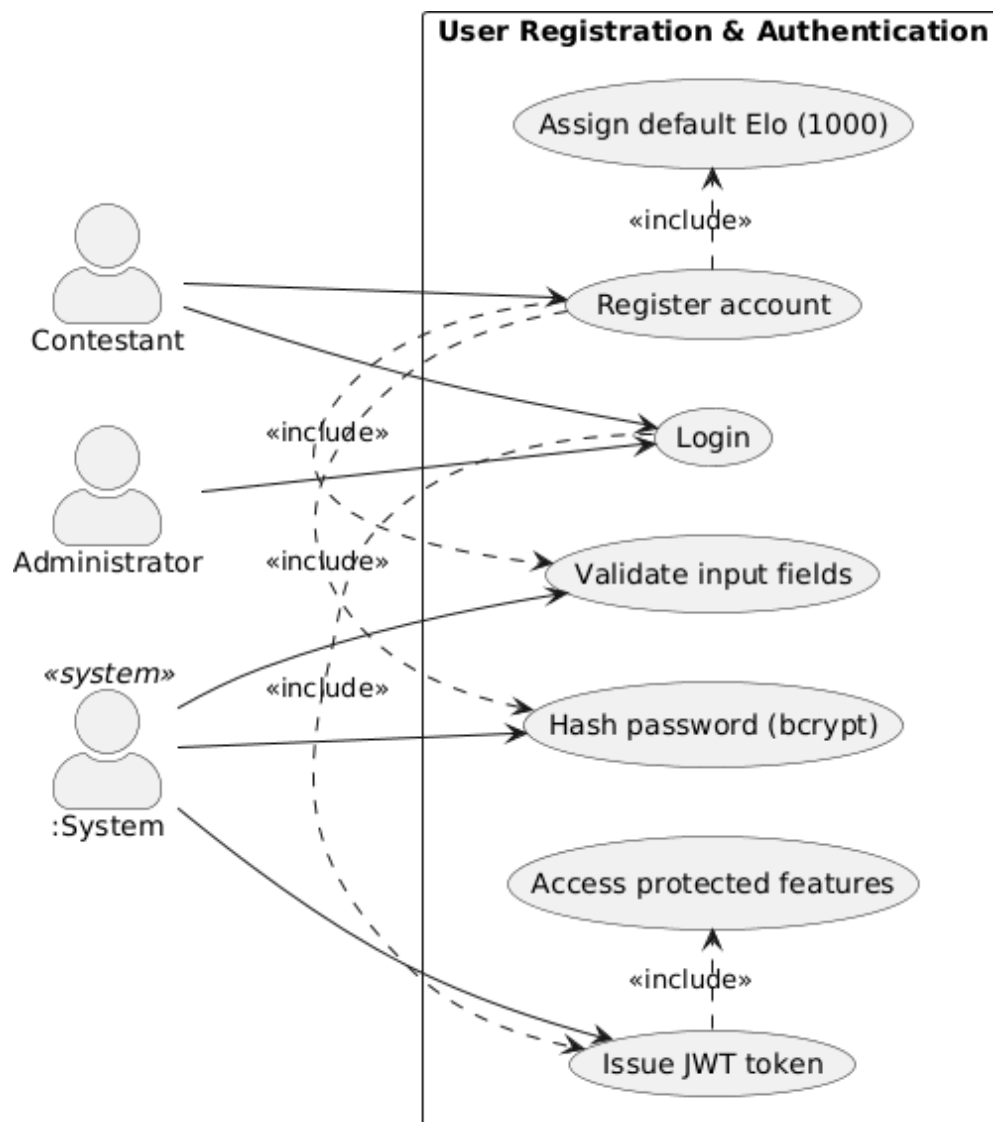


Figure 2: Use cases — user registration and authentication (FR-1)

4.2.2 Written Use Cases

UC-1.1: Register Account

Actor	Contestant
Precondition	The user is not logged in and has no existing account.
<i>Main success scenario</i>	
1	The user navigates to the registration screen.
2	The user enters a username, email address, and password.
3	The system validates that the email format is correct, the username is between 3 and 20 alphanumeric characters, and the password is at least 8 characters.
4	The system verifies that neither the email nor the username is already registered.
5	The system hashes the password with bcrypt (cost factor ≥ 10) and creates a new user record with the CONTESTANT role and an initial Elo rating of 1000.
6	The user is redirected to the login screen with a confirmation message.
<i>Alternate flows</i>	
3a	If any field fails format validation, the system displays an inline error message and the user record is not created.
4a	If the email or username is already taken, the system returns HTTP 409 with a descriptive message identifying which field conflicts.

UC-1.2: Login

Actor	Contestant or Administrator
Precondition	The user has a registered account and is not currently logged in.
<i>Main success scenario</i>	
1	The user navigates to the login screen.
2	The user enters their email address and password.
3	The system verifies the password against the stored bcrypt hash.
4	The system issues a JWT token with a 24-hour expiry and returns it to the client.
5	The user is redirected to the dashboard.
<i>Alternate flows</i>	
3a	If the credentials are invalid, the system returns HTTP 401 with the generic message "Invalid email or password." No information about whether the email exists is disclosed.

4.3 Matchmaking (FR-2)

4.3.1 Diagram

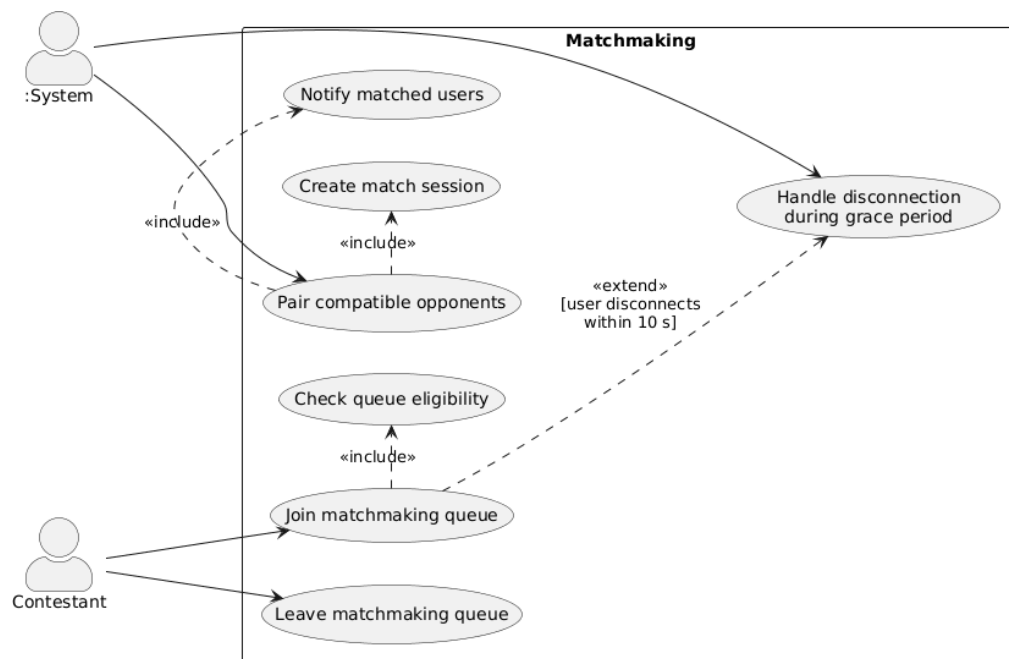


Figure 3: Use cases — matchmaking (FR-2)

4.3.2 Written Use Case

UC-2.1: Join Matchmaking Queue and Be Paired

Actor	Contestant
Precondition	The contestant is logged in, is not currently in the queue, and is not in an active match.
<i>Main success scenario</i>	
1	The contestant clicks the “Find Match” button on the dashboard.
2	The system verifies that the contestant is not already queued or in an active match.
3	The system adds the contestant to the in-memory matchmaking queue, recording their current Elo rating and a join timestamp.
4	The system scans the queue and finds another contestant whose Elo rating is within 200 points.
5	The system creates a match session with status ACTIVE, selects a problem from the approved pool that neither contestant has seen before, and records both user IDs, the problem ID, and the start timestamp.
6	Both contestants receive a WebSocket notification containing the match session identifier and are redirected to the match screen.
<i>Alternate flows</i>	
2a	If the contestant is already queued or in a match, the system returns HTTP 409 and no queue entry is created.
4a	If no compatible opponent is found within the current scan cycle, the contestant remains in the queue and the system retries on the next scan.
6a	If one of the two matched contestants disconnects within the 10-second grace period after notification, the match is cancelled, the remaining contestant is returned to the queue, and the disconnecting contestant’s session is terminated.

4.4 Match Session and Code Evaluation (FR-3, FR-4)

4.4.1 Diagram

Figure 4 combines the match session (FR-3) and automated code evaluation (FR-4) use cases, reflecting their tight coupling at runtime. The Judgeo API actor represents the external sandboxed execution service; it is invoked exclusively by the backend.

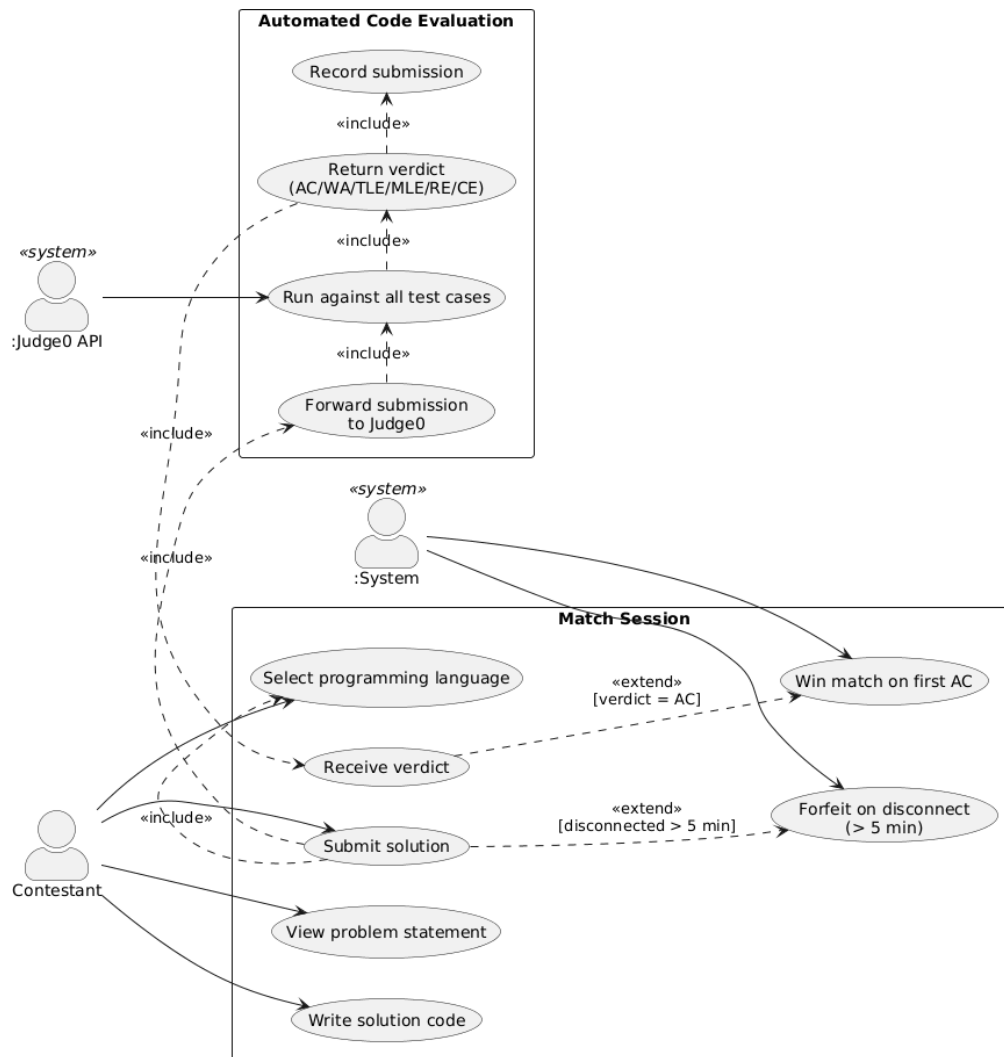


Figure 4: Use cases — match session and automated code evaluation (FR-3, FR-4)

4.4.2 Written Use Case

UC-3.1: Submit Solution During Match

Actor	Contestant
Precondition	The contestant is in an active match session. The match timer has started and has not yet expired.
<i>Main success scenario</i>	
1	The contestant writes or pastes source code into the editor and selects a programming language.
2	The contestant clicks the “Submit” button.
3	The system forwards the source code, language identifier, and problem resource limits to the Judgeo API for each test case defined for the problem.
4	Judgeo executes the submission against each test case in an isolated sandbox and returns per-test-case results.
5	The system compares each test case output (trimmed of trailing white-space) against the expected output and records the number of test cases passed.
6	If all test cases pass, the verdict is AC. The system marks the match as COMPLETED, declares the submitting contestant the winner, triggers the rating update (FR-5), and sends a WebSocket notification to both contestants. Both are redirected to the post-match screen.
7	If not all test cases pass, the verdict (WA, TLE, MLE, RE, or CE) is displayed to the submitting contestant. The match continues.
<i>Alternate flows</i>	
3a	If the Judgeo API is unavailable, the submission is rejected with the message “Execution service temporarily unavailable.” The match is not voided; the contestant may resubmit.
4a	If Judgeo returns an unexpected error, the submission is marked JUDGE_ERROR and the contestant is notified. The submission does not count toward test case totals.
6a	If the match timer expires before any AC verdict is received, the system compares total test cases passed across all submissions by both contestants. The contestant with the higher count wins. In the event of a tie, the match is recorded as a draw. Rating updates are applied in either case.
6b	If a contestant disconnects and does not reconnect within 5 minutes, they forfeit the match. The opponent is declared the winner and rating updates are applied.

4.5 Rating System (FR-5)

4.5.1 Diagram

Figure 5 covers the Elo rating update lifecycle. All steps in this diagram are system-triggered following match completion; the contestant’s only direct interaction is viewing the resulting rating change on the post-match screen.

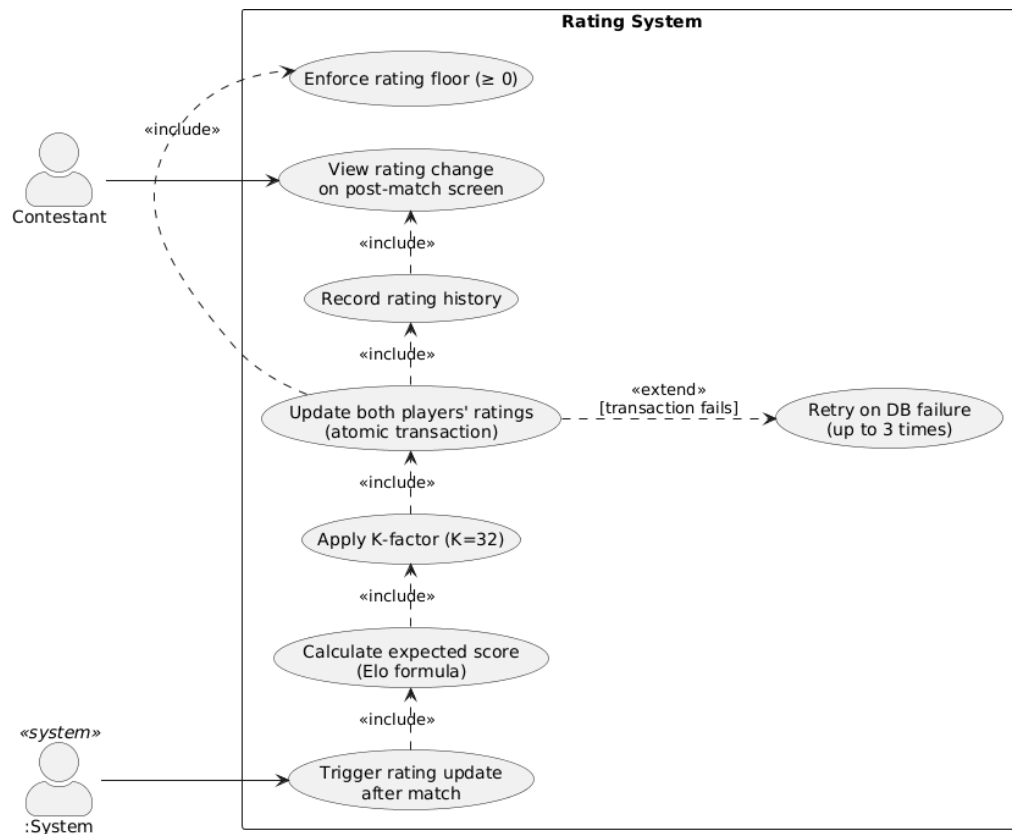


Figure 5: Use cases — rating system (FR-5)

4.5.2 Written Use Case

UC-5.1: Update Ratings After Match

Actor	System (triggered automatically on match completion)
Precondition	A match record has been finalized with status COMPLETED and a definitive outcome (WIN_A, WIN_B, or DRAW).
<i>Main success scenario</i>	
1	The system retrieves the current Elo ratings of both contestants from the database.
2	The system calculates the expected score for each contestant using the Elo formula: $E_A = 1/(1 + 10^{(R_B - R_A)/400})$.
3	The system calculates the new rating for each contestant: $R' = R + 32 \cdot (S - E)$, where S is 1 (win), 0.5 (draw), or 0 (loss).
4	If the calculated new rating for either contestant is below 0, it is clamped to 0.
5	The system persists both updated ratings and inserts a RatingHistory record for each contestant within a single atomic database transaction.
6	The rating delta for each contestant is included in the post-match screen payload delivered via WebSocket.
<i>Alternate flows</i>	
5a	If the database transaction fails, the system retries up to three times. If all retries fail, the error is logged and flagged for manual review. The match record remains COMPLETED.

4.6 Problem Proposal and Approval Workflow (FR-6)

4.6.1 Diagram

Figure 6 captures the community-driven problem contribution model. The administrator's ability to edit a proposal before deciding is modelled as an «extend» relationship to both the approve and reject use cases, reflecting that editing is optional.

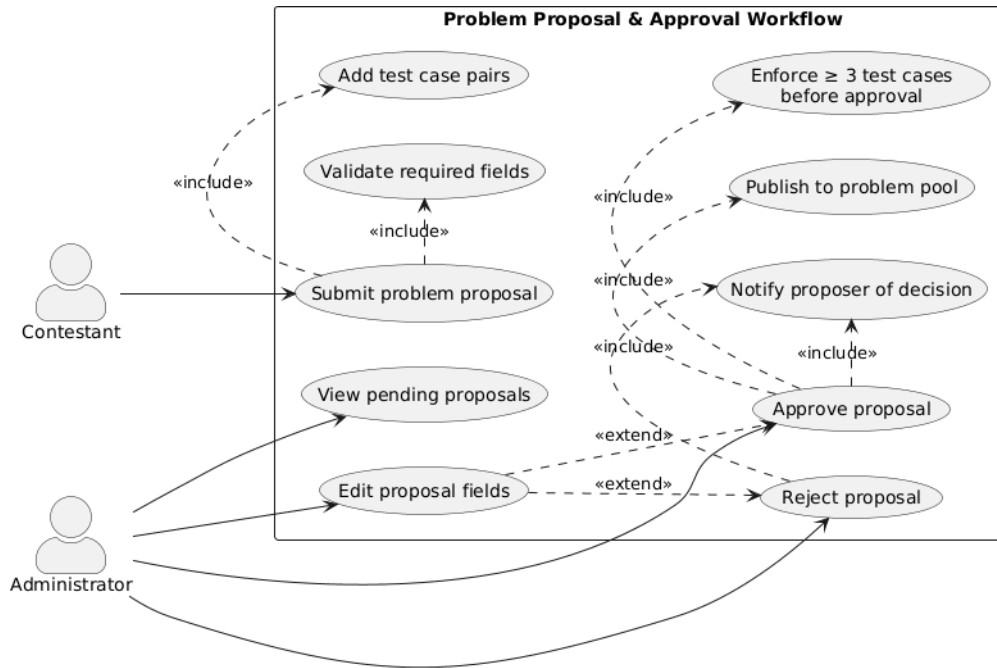


Figure 6: Use cases — problem proposal and approval workflow (FR-6)

4.6.2 Written Use Case

UC-6.1: Submit Problem Proposal

Actor	Contestant
Precondition	The contestant is logged in.
<i>Main success scenario</i>	
1	The contestant navigates to the problem proposal form.
2	The contestant fills in the problem title, statement, input and output format descriptions, constraints, time limit, memory limit, and at least one test case pair. An optional difficulty tag may be selected.
3	The system validates that all required fields are present and within their defined bounds.
4	The system stores the proposal with status PENDING, recording the proposing user's ID and the submission timestamp.
5	The contestant is shown a confirmation that the proposal has been submitted for review.
<i>Alternate flows</i>	
3a	If any required field is missing or out of bounds, the frontend displays inline validation errors and the proposal is not submitted.

UC-6.2: Review and Decide on Problem Proposal

Actor	Administrator
Precondition	At least one problem proposal with status PENDING exists. The administrator is logged in with the ADMIN role.
<i>Main success scenario (approve)</i>	
1	The administrator navigates to the administrator panel and views the list of PENDING proposals sorted by submission date.
2	The administrator selects a proposal and reviews its fields.
3	(Optional) The administrator edits one or more fields of the proposal.
4	The administrator verifies that the proposal contains at least three test case pairs. The approve button is enabled only when this condition is met.
5	The administrator clicks “Approve.”
6	The system sets the proposal status to APPROVED, adds the problem to the active pool, and sends an in-system notification to the proposing contestant.
<i>Alternate flow (reject)</i>	
5a	The administrator clicks “Reject,” optionally entering a rejection reason.
6a	The system sets the proposal status to REJECTED, stores the rejection reason, and sends an in-system notification to the proposing contestant. The problem is not added to the pool.
<i>Exception flows</i>	
4a	If the proposal has fewer than three test cases, the approve button is disabled. The administrator must either add test cases in step 3 or reject the proposal.
–	If a non-administrator attempts to access the approval endpoints, the system returns HTTP 403 and the action is not performed.

4.7 Leaderboard and Match History (FR-7)

4.7.1 Diagram

Figure 7 covers the read-only visibility features. The leaderboard is publicly accessible to unauthenticated guests, while match history is private and restricted to the authenticated owner.

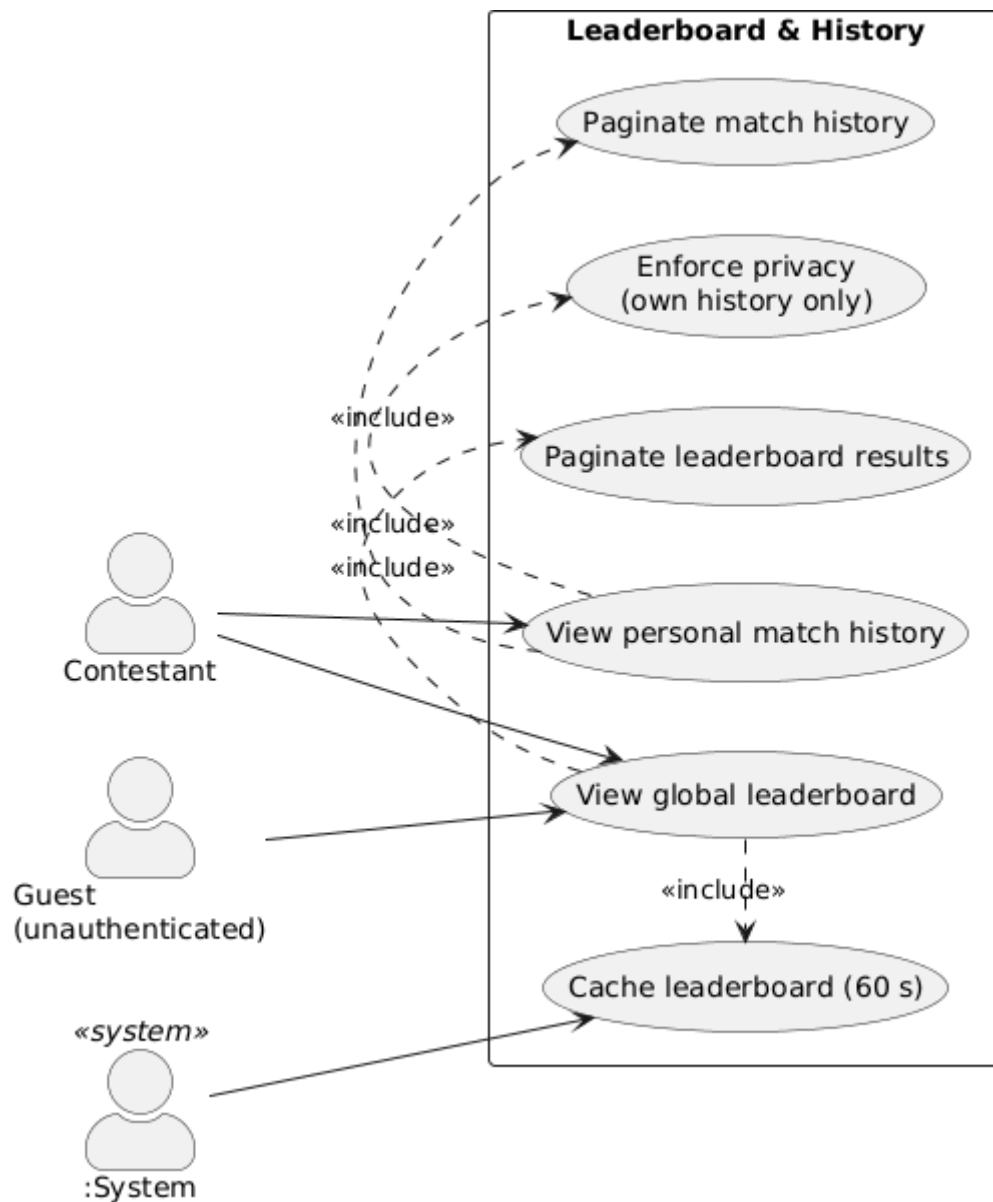


Figure 7: Use cases — leaderboard and match history (FR-7)

4.7.2 Written Use Cases

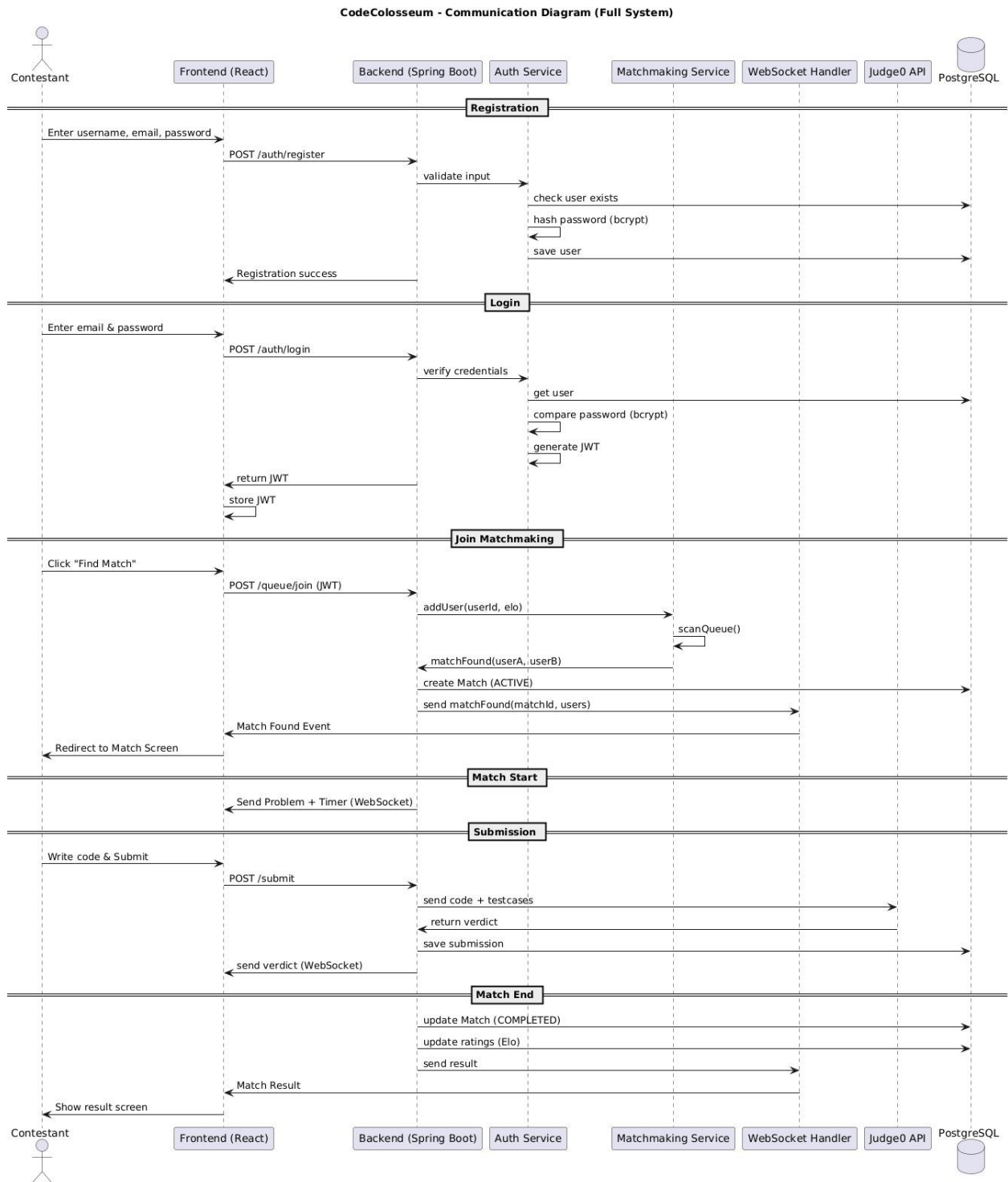
UC-7.1: View Global Leaderboard

Actor	Guest or Contestant
Precondition	None. Authentication is not required.
<i>Main success scenario</i>	
1	The user navigates to the leaderboard screen.
2	The system checks whether a cached leaderboard result exists and is less than 60 seconds old.
3	If no valid cache exists, the system queries all users ordered by current Elo rating descending and caches the result.
4	The system returns the first page of results (50 users per page), showing rank, username, current Elo rating, and total matches played.
5	The user may navigate to subsequent pages using pagination controls.
<i>Alternate flows</i>	
3a	If the database is unavailable, the system returns HTTP 503 and displays an appropriate error message to the user.

UC-7.2: View Personal Match History

Actor	Contestant
Precondition	The contestant is logged in with a valid JWT token.
<i>Main success scenario</i>	
1	The contestant navigates to their match history page.
2	The system extracts the contestant's user ID from the JWT token.
3	The system retrieves all match records associated with that user ID, ordered by match timestamp descending.
4	The system returns a paginated response showing, for each match, the opponent's username, the problem title, the outcome (win/loss/draw), the Elo delta, and the match timestamp.
5	The contestant may navigate to subsequent pages using pagination controls.
<i>Alternate flows</i>	
2a	If the JWT token is missing or invalid, the system returns HTTP 401 and redirects the user to the login screen.
2b	If the request targets another user's history, the system returns HTTP 403 and no records are returned.
3a	If the database is unavailable, the system returns HTTP 503 and displays an appropriate error message to the user.

5 System Sequence Diagram



A Appendices

The following decision log entries document key engineering decisions made during requirements analysis. These entries are required by the project guidelines and are considered part of this SRS.

A.1 Decision Log: Code Execution Strategy

Field	Detail
Decision	Delegate all code execution to the Judgeo REST API in the initial implementation.
Alternatives considered	(1) Build a custom Docker-based execution runner from scratch. (2) Use Judgeo API (selected).
Rationale	Building a custom runner with adequate sandboxing, multi-language support, and resource limiting would consume a disproportionate share of the available development time. Judgeo provides a well-tested, production-grade sandbox with broad language support and a straightforward REST interface.
Trade-offs	Introduces an external runtime dependency. System availability for match functionality is partially contingent on Judgeo uptime and rate limits.
Risk mitigation	The execution logic is encapsulated behind a JudgeService interface (NFR-11). If Judgeo becomes unavailable or its terms change, only one implementation class needs to be replaced.
Technical debt	The custom Docker-based runner remains a future option and is documented as a candidate for implementation in a subsequent iteration.

A.2 Decision Log: Rating Algorithm

Field	Detail
Decision	Use the standard Elo algorithm with a fixed K-factor of 32 for all rating updates.
Alternatives considered	(1) Simple win/loss points system (e.g., +10 / -10 per match). (2) Glicko-2 rating system. (3) Standard Elo with K=32 (selected).
Rationale	The simple points system does not account for opponent strength, making it unfair and unrepresentative of actual skill. Glicko-2 adds rating deviation and volatility parameters which improve accuracy but significantly increase implementation complexity. Standard Elo is well-understood, widely used in competitive contexts, straightforward to implement correctly, and produces meaningful and interpretable ratings.
Trade-offs	Elo with K=32 can produce volatile rating swings for new users with few matches. It does not account for rating reliability (unlike Glicko-2).
Risk mitigation	The rating logic is encapsulated in a single RatingService class. The K-factor is a configurable parameter, not a hard-coded constant, allowing adjustment without code changes.
Technical debt	Migrating to Glicko-2 is documented as a future improvement if the platform grows beyond the academic context.

A.3 Decision Log: Match Winner Determination

Field	Detail
Decision	In the event that neither contestant achieves an AC verdict before time expires, the winner is determined by the number of test cases passed across all submissions. In the case of a tie, the match is recorded as a draw.
Alternatives considered	(1) First contestant to submit any AC wins; otherwise the match is always a draw. (2) Partial scoring as described above (selected). (3) Penalty system.
Rationale	A pure “first AC wins or draw” rule would penalize contestants who make significant partial progress but are narrowly beaten or encounter a difficult edge case. Partial test case scoring rewards productive effort and reduces the frequency of uninformative draws.
Trade-offs	Partial scoring requires tracking and comparing per-submission test case pass counts, adding modest complexity to match resolution logic. Wrong Answers are also not penalized at all, making the resulting conclusion less accurate for both sides.
Risk mitigation	Low. The resolution logic is isolated to the match service and does not affect any other component.

Acknowledgment: The structure of this document is based upon the IEEE Guide to Software Requirements Specification (ANSI/IEEE Std 830-1984) and the SRS template provided for CSE3044 Software Engineering, Marmara University.